

SEOS-3215 TLS Server Functionality in os_tls / if_OS_Tls

Page Content

- [Jira Tickets](#)
- [Current State](#)
 - [Structure](#)
 - [Architecture](#)
- [Simplified Structure](#)
 - [Architecture](#)
- [Integration Analysis](#)
 - [Demo TLS Server](#)
 - [Configuration](#)
 - [Functions](#)
 - [TLS API](#)
 - [Enums](#)
 - [Structs](#)
 - [Functions](#)
- [Summary](#)
 - [Configuration](#)
 - [Functions](#)
 - [Build](#)
- [Next Steps](#)

Jira Tickets

Concept	SEOS-3215 - Getting issue details...STATUS
Next step	SEOS-3237 - Getting issue details...STATUS

Current State

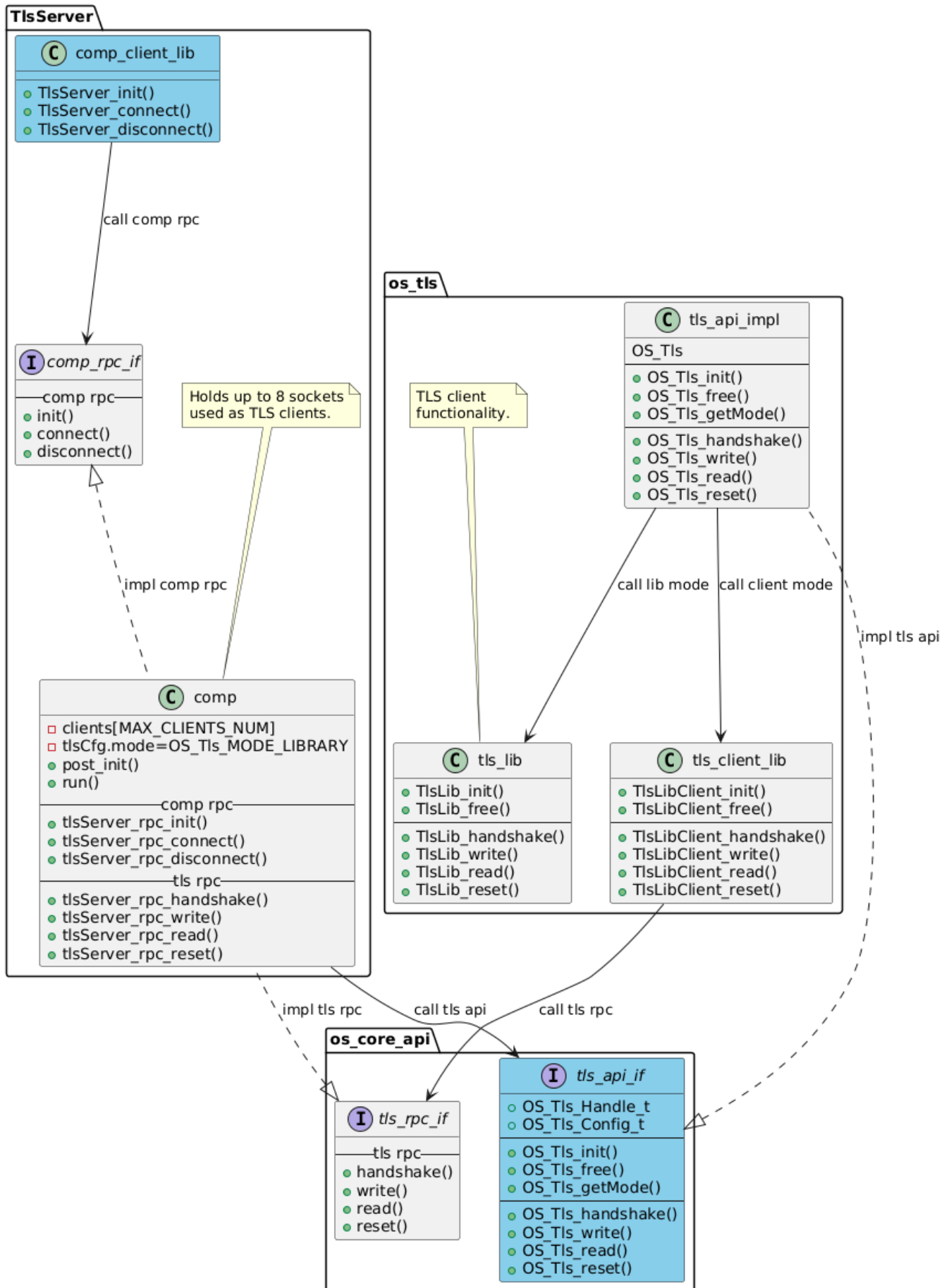
This chapter shows the current structure (files, CMake targets) and architecture of the SDK.

Structure

- **os_core_api (*)**
 - TLS API (tls_api_if)
 - OS_Tls.h
 - TLS interface (tls_rpc_if)
 - interfaces/if_OS_Tls.h
 - camkes/if_OS_Tls.camkes
 - camkes/if_OS_Tls.camkes.h
- **os_tls (*)**
 - TLS API (tls_api_impl)
 - OS_Tls.c
 - TLS lib (tls_lib)
 - inc/lib/TlsLib.h
 - src/lib/TlsLib.c
 - TLS RPC lib (tls_client_lib)
 - inc/rpc/TlsLibClient.h
 - src/rpc/TlsLibClient.c
- TlsServer
 - **TlsServer (comp) (*)**
 - server/src/TlsServer.c
 - camkes/TlsServer.camkes
 - **TlsServer_client (comp_client_lib) (*)**
 - client/include/TlsServer_client.h
 - client/src/TlsServer_client.c
 - TlsServer interface (comp_rpc_if)
 - client/include/if_TlsServer.h
 - camkes/if_TlsServer.camkes

(*) : CMake targets

Architecture



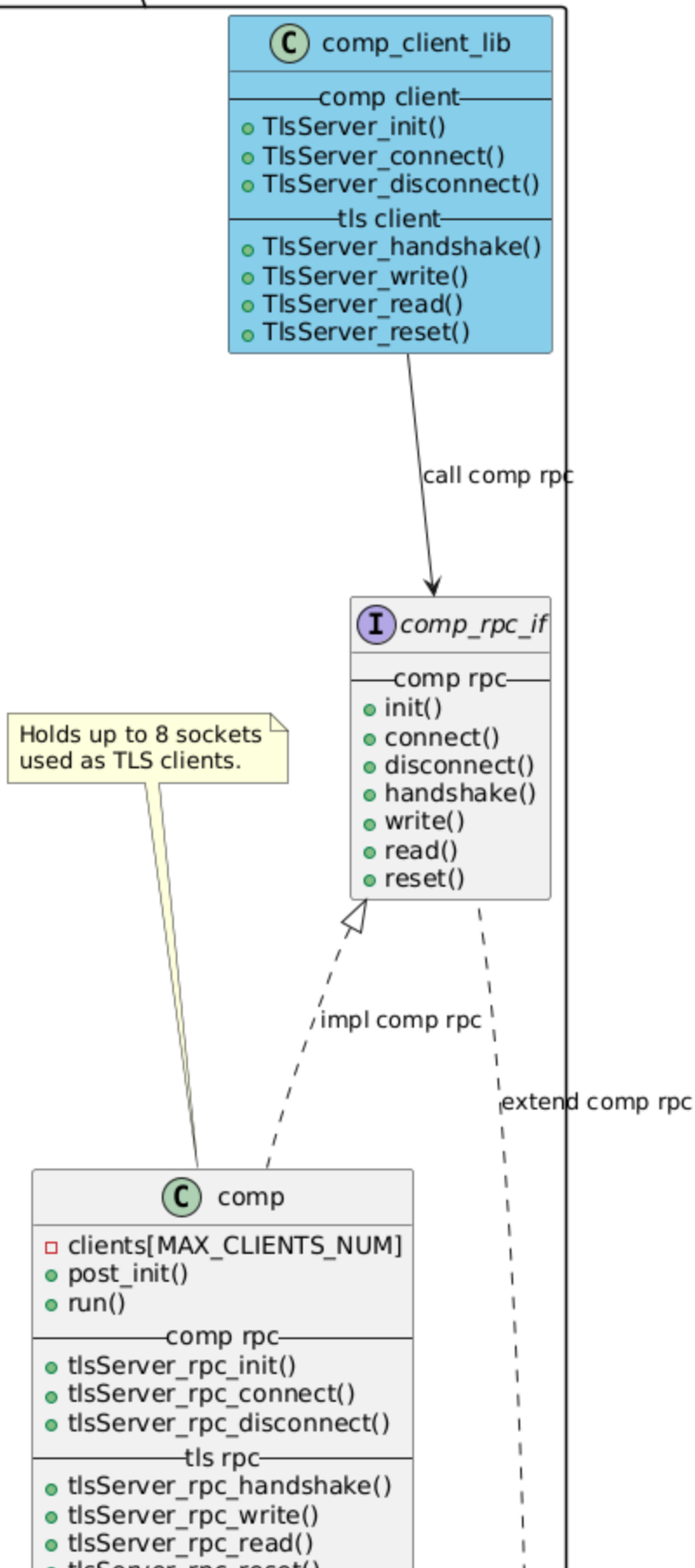
Simplified Structure

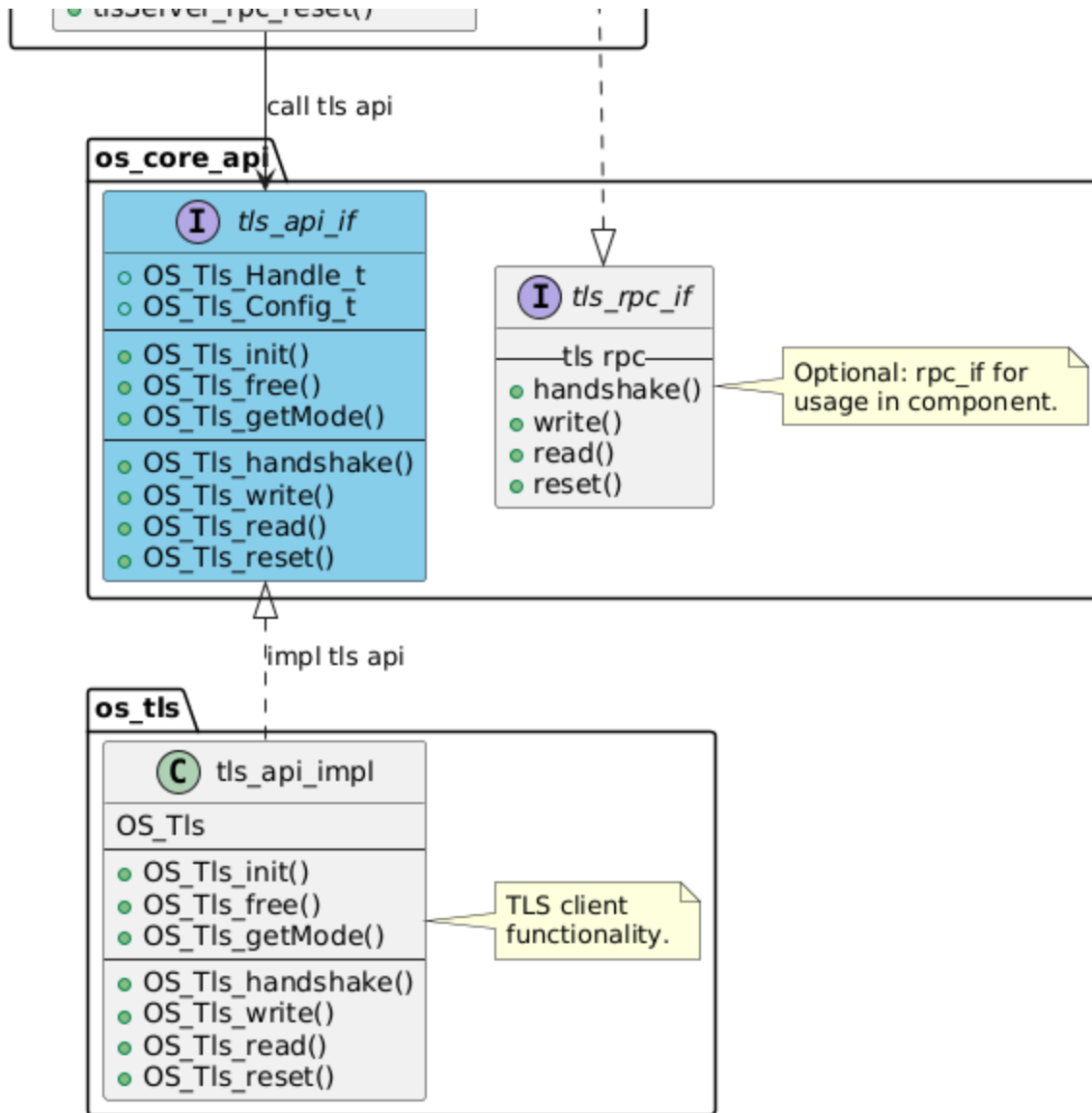
Without the "client library" of `os_tls` the complexity could be reduced:

- Remove `tls_client_lib` in `os_tls`.
- Remove `tls_lib` and implement TLS functionality in `tls_api_impl`.
- Implement client functionality in `comp_client_lib`.
- Optionally: Provide `tls_rpc_if` in `os_core_api` to extend `comp_rpc_if`.

Architecture

TlsServer





Integration Analysis

Demo TLS Server

Configuration

- mbedtls_debug_set_threshold: debug level
- mbedtls_x509_cert_parse: server root cert (PEM or DER format, `char[]`)
- mbedtls_x509_cert_parse: server cert (PEM or DER format, `char[]`)
- **mbedtls_pk_parse_key: server private key (PEM or DER format, `char[]`)**
- mbedtls_ctr_drbg_seed: entropy function callback
- mbedtls_ssl_config_defaults:
 - IS_SERVER,
 - TRANSPORT_STREAM,
 - PRESET_DEFAULT
- mbedtls_ssl_conf_authmode:
 - VERIFY_REQUIRED (server default)
 - VERIFY_NONE (client default)
 - VERIFY_OPTIONAL (check but do not fail)
- mbedtls_ssl_conf_ciphersuites: 0 terminated list of IDs
- mbedtls_ssl_conf_rng: random number generator callback
- mbedtls_ssl_conf_dbg: debug log function callback
- mbedtls_ssl_conf_ca_chain: configure server trusted certs
- **mbedtls_ssl_conf_own_cert: configure server certificate and private key**

- mbedtls_ssl_setup

Functions

- mbedtls_ssl_session_reset
- mbedtls_ssl_set_bio
- mbedtls_ssl_handshake
- mbedtls_ssl_read
- mbedtls_ssl_write
- mbedtls_ssl_close_notify

TLS API

Enums

- OS_Tls_Mode_t:
 - LIBRARY
 - CLIENT
- OS_Tls_Digest_t, OS_Tls_Digest_Flags_t:
 - SHA256
- OS_Tls_CipherSuite_t, OS_Tls_CipherSuite_Flags_t:
 - OS_Tls_CIPHERSUITE_DHE_RSA_WITH_AES_128_GCM_SHA256
 - OS_Tls_CIPHERSUITE_ECDHE_RSA_WITH_AES_128_GCM_SHA256
- OS_Tls_Flag_t:
 - OS_Tls_FLAG_DEBUG
 - OS_Tls_FLAG_NO_VERIFY
 - OS_Tls_FLAG_NON_BLOCKING

Structs

- OS_Tls_Policy_t:
 - OS_Tls_Digest_Flags_t: Handshake digests
 - OS_Tls_Digest_Flags_t: Certificate digests
 - RSA min bit length
 - DH min bit length
- TlsLib_Config_t:
 - socket:
 - recv callback
 - send callback
 - context
 - crypto:
 - OS_Tls_Policy_t
 - OS_Crypto_Handle_t
 - caCerts ('char[]')
 - OS_Tls_CipherSuite_Flags_t
 - OS_Tls_Flag_t
- OS_Tls_Config_t:
 - OS_Tls_Mode_t
 - TlsLib_Config_t
 - if_OS_Tls_t

Functions

- OS_Tls_init()
- **OS_Tls_handshake() (*)**
- **OS_Tls_write() (*)**
- **OS_Tls_read() (*)**
- **OS_Tls_reset() (*)**
- OS_Tls_free()
- OS_Tls_getMode()

(*) : RPC interface

Summary

Configuration

1. From a configuration point of view the main difference between TLS API and Demo TLS Server is that Demo TLS Server requires an own certificate and a private key in addition to the trusted certificates.
2. The own certificate and private key is required for the authentication (per default a client verifies the authenticity of the server).
3. At the moment the TLS API does only support an TLS client without authentication (server cannot authenticate the client).
4. Hence, also for a authentication of a TLS client an own certificate and a private key would be required.

Functions

1. Besides this the API functions should be sufficient also for a TLS server.
2. The closing / reset of a TLS server session needs to be examined.

Build

1. At the moment for TLS server or TLS client different mbedtls libraries have to be used because `os_crypto` is not yet used by the mbedtls TLS server functionality.
2. Hence, only a compile-time selection via CMake target and/or compile-time defines can be supported (no configuration via configuration structs).

Next Steps

1. The expected number of necessary changes for the integration of TLS server functionality looks limited.
2. The chance that the TLS API stays stable is high.
3. API extensions might be required for authentication anyways.
4. The next step is to integrate the TLS server functionality into the existing architecture.